

Programming in R: Exercise 2

Felix Clouth, f.j.clouth@tilburguniversity.edu

General remarks & tips

- Write your code in an .R script
- Don't save your workspace; start "clean" by running `rm(list = ls())`
- Add comments to your code using `#`
- Don't use `=` to assign values, use `<-`
- A good summary "cheat sheet" of data types and how to index them, can be found here: <https://www.statmethods.net/input/datatypes.html>

Part 1: Types of Data

1. Matrices.

- a. Create the following matrix and save it in an object (note that this matrix is different from the matrix that I showed in the slides, check the help page of the function `matrix()` to check which argument can be added to the code from the lecture to obtain the matrix that you need):

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

- b. From your matrix, select the element on the second row and in the third column.

2. Data frames.

- a. Execute (when I say "execute", I mean "run in R"):

```
llama.agreeableness <- c(50, 20, 15, 80, 35, 52)
llama.age <- c(14, 9, 10, 25, 11, 15)
llamas <- data.frame(llama.age, llama.agreeableness)
```

Make R display only the llama's agreeablenesses by using the `$` operator.

- b. The zookeeper subjected the llamas to an experiment to see what the effect of listening to heavy metal is on their agreeableness. Using the `$` operator, add a new column to the `llamas` data frame called `metal` that has the values 1 and 2

repeated 3 times (i.e., the first llama is in condition 1, the second in condition 2, the third in condition 1, etc.). Hint: use the function `rep()`.

- c. After completing question b, execute:

```
llamas$metal <- factor(llamas$metal)
levels(llamas$metal) <- c("No", "Yes")
```

What does this do?

- d. Execute:

```
llama.names <- c("Arthur", "Zaphod", "Trillian", "Ford", "Marvin",
"Rooster")
row.names(llamas) <- llama.names
```

What does this do?

- e. Using the data frame you created, make R show only the third llama.
f. Using the data frame you created, make R show only the first and third llamas.
g. Using the data frame you created, make R show only Zaphod and Rooster.
3. Creating different types of data.
- a. Create a matrix with the latest grades of 3 students (rows), for two different courses (columns).
b. Create a data frame with data from the students sitting next to you. Include the variables name, eyecolor, gender, and height.
c. Create a list with the name of our course, the matrix, and the data frame you just made.
4. Execute:

```
StarWars <-
list(series = "starwars",
      movie_order = c(4, 5, 6, 1, 2, 3, 7, 8, 9),
      characters = data.frame(
        name = c("Luke", "C-3PO", "R2-D2", "Darth Vader", "Leia"),
        height = c(172, 167, 96, 202, 150),
        evil = c(FALSE, FALSE, FALSE, TRUE, FALSE)))
```

- a. Change the value of the object series from starwars to Star Wars.
b. Use square brackets to extract the position in the movie order of the third movie from the list.
c. Add a factor variable indicating the species of the characters to the data frame within the list (human, droid, droid, human, human).

5. We forgot to add Chewbacca to the data frame with the StarWars characters from question 5. He is 228cm tall and definitely not evil. We forgot what species he is, though¹, and we'll consider it a missing value. Add a row to the character data frame in the starwars list with Chewbacca's details.
-

Part 2: Working with Data

6. Consider the data frame you created in question 2.
 - a. Using the data frame you created, make R show only the data for the llamas above age 11. Hint: use `>` in a selection.
 - b. Now make R show only the other llamas (i.e., age 11 or below). Try to achieve this in two different ways.
 - c. Make R show only the llamas above age 11 who are also in the `metal == "Yes"` condition. Hint: use `&`.
 - d. Make R show only the llamas who are either above age 11 or in the `metal == "Yes"` condition.
7. There are several data sets available in R. For example, Motor Trend Car Road Tests: `mtcars`.
 - a. Save the data set `mtcars` in your workspace as an object.
 - b. A quick way to inspect such data sets is with the functions `str()`, `summary()`, `head()`, and `View()`. What does each of these functions do?
 - c. How many cars in this data set have at least 6 cylinders? Hint: datasets that are available in R have help pages just as the functions do, which contain details on what is included in the dataset, these pages can be accessed in the same way as the help pages for the functions.
 - d. What is the average weight of the cars in this data set?
 - e. What is the average gross horsepower of the cars that have fewer than 6 cylinders?
8. Some data sets are only available in specific R packages.
 - a. Install & activate the package `dplyr`, and save the data set `starwars` in an object.
 - b. How many variables are there in this data set?
 - c. If I would stack all characters on top of each other, how tall would the stack be?
 - d. Who is the tallest character? And who the heaviest?

¹ Of course we didn't forget. Obviously, Chewbacca is a Wookiee.

9. You can also load data sets from separate files. You can read in data from a file in different ways:

Option 1: specifying the path -----

*# I can tell R the exact location of where my file is
Note that R uses forward slashes (/) instead of backslashes (\) to
separate directories, so when copy-pasting the path you will need to
change them*

```
data <- read.table("C:/Felix/Desktop/my_file.txt", header = TRUE)
```

Option 2: setting your work directory -----

I can first set the working directory to the location of my file

```
setwd("C:/Felix/Desktop")
```

now R will search this folder if I tell it to look for a file

```
data <- read.table("my_file.txt", header = TRUE)
```

Option 3: file.choose() -----

I can use a pop-up window where I can choose my file

```
my_file <- file.choose()
```

*# this will give a pop-up window, in which you can select the data file
note: sometimes you have to use Alt + Tab to find it*

*# the object "my_file" now contains the path to the file that you have
chosen*

you can check what the path is by calling the object again:

```
my_file
```

now you can use this object to read in data

```
data <- read.table(my_file, header = TRUE)
```

*The data for this exercise are originally from
<https://github.com/ayoubimaya/pokemon>.*

The data files referred to in this question can be found on Canvas > Modules > Week 2.

- a. Download and save the file `pokemon.txt`. Read in the data in R using `read.table()`. *Tip: don't forget to specify the argument header.*
 - b. Download and save the file `pokemon.csv`. Read in the data in R using `read.csv()`. Check what your data looks like with `head()`. Next, read in the data using `read.csv2()` and check your data again with `head()`. Which of the 2 functions do you need? What is the difference?
 - c. Download and save the file `pokemon.RData`. Which function do you need to read in these data? How is this different from the commands you had to use in a & b?
 - d. Download and save the file `pokemon.sav`. Read in the data in R using `import()` from the package `rio`. Make sure that you install and activate the package first.
 - e. Can you also use `import()` to read in the `.txt`, `.csv`, and `.RData` files?
10. Save one of the pokemon data sets in the object `pokemon_data`.
 - a. Create a new data frame with only "Water" pokemon.
 - b. Save your new data frame as a comma separated file. Note: I don't want to save the row numbers.
 - c. Save your new data frame as an `.RData` file.
11. From Canvas>Modules>Week 2, download the paper "How Smart Do You Think You Are? A Meta-Analysis on the Validity of Self-Estimates of Cognitive Ability" by Freund & Kasten (2011). Read the abstract and Table 1.
 - a. Download and save the data set `smart.csv` from Canvas>Modules>Week2. This dataset corresponds to Table 1 in the paper (I only added the last column, `z.se`, which is the standard error of the z-transformed effect size).
 - b. Read in the `smart.csv` data set using the `read.csv()` function and assign the result to a variable `smart`.
 - c. On page 13 of the paper, it reads:

The **154** effect sizes ranged from $r = -.25$ to $r = .85$, with a mean of **.32** and a standard deviation of **.19**. Over **93%** (i.e., **144** of the **154**) of the effect sizes were positive, indicating initial support for the first hypothesis, that the relationship between self-estimated and psychometrically assessed cognitive abilities is positive. The accumulated sample size for these **154** effect sizes was **22,256**, which corresponds to an average sample size of about **145** participants per study (the minimum sample size was **13**, and the maximum sample size was **1,508**).

Reproduce all of the numbers in this paragraph (shown in **bold**) by using R commands.
12. Use the `starwars` data in the package `dplyr` to make a table of the different hair colors, but only for characters who have a star ship.

13. The function `import()` from the package `rio` can have some hiccups (if the function encounters strange things in your data file, it will sometimes only load half of it, without giving a warning). Instead, read in the Excel file from question 9 with the function `read_excel()` from the package `readxl`, and the SPSS file with the function `read.spss()` from the package `foreign`. Make sure that you have installed and activated the required packages first.
14. Indexing lists (and data frames) correctly can be tricky. In the tweet below, Hadley Wickham (R guru) explains the difference between different methods of indexing lists:



Indexing lists

- a. See if you can figure out what the difference is between these methods. First, create a small data frame (which is a special case of a list):

```
df <- data.frame(a = 1:5,  
                 b = 6:10)
```

Next, try these different methods of indexing. What are the differences? Hint: use `typeof()`.

```
df[1]
df[[1]]
df[[1]][[1]]
```

b. Now, consider these forms of indexing:

```
df$a
df[, 1]
```

Which of the different methods of indexing in 14a do these two methods resemble most?

15. In this exercise we want to check whether for our dataset manual cars can go more miles per gallon than automatic cars. Run the following code to load the dataset for this exercise (Note: it is based on the `mtcars` dataset in R)

```
mtcars_abdridged <- data.frame(mpg = mtcars$mpg,
                               disp = mtcars$disp,
                               hp = mtcars$hp,
                               vs = mtcars$vs,
                               am = ifelse(mtcars$wt %in% c(5.424,
2.320, 1.935, 4.070), NA, mtcars$am),
                               gear = mtcars$gear)

names(mtcars_abdridged)[1] <- "miles per gallon"
```

- Transform the transmission type variable into a factor with informative labels. Set manual as the first level in the factor.
- How many cars in the dataset are automatic, how many are manual, and for how many is this information missing?
- Run a t-test using the function below, making sure that the correct data is inserted in place of the names between `<>`, and store the result.

```
t.test(<miles per gallon data manuals>, <miles per gallon data
automatics>, var.equal = TRUE)
```

- Run the code below, making sure that the `t_test_df`, `t_test_t-statistic`, & `t_test_p-value` are correctly referenced from the t-test's list of results.

```
sprintf("t(%i) = %f, p = %f", t_test_df, t_test_t_statistic,
t_test_p_value)
```

- Using an alpha cutoff value of .05, can manual cars in our dataset go statistically significantly more miles per gallon than automatic cars?

Extra questions

Many people prefer to use the functionality of the `dplyr` package. The following questions are not part of this course, however, they might be useful for you to know.

16. Get familiar with the package `dplyr`

- a) What is the function of the `%>%` operator? How is it different from how base R works?
- b) Some very useful functions are `filter()`, `select()`, `group_by()`, and `summarise()`. What do these functions do?
- c) Answer Question 7 c – e using functions from the `dplyr` package.
- d) Create a new variable “power to weight ratio” (divide horsepower of the engine by weight of the car). Which function from the `dplyr` package is essential for this to work?